Lab Deep Dives 2 — Refactoring, Actualization, Testing, BDD

> Per-lab depth for the four exam-heaviest labs of SB5-MAI: the Refactoring Lab (Lecture 4), the Actualization Lab (Lecture 5), the Testing Lab / TestLab1 (Lecture 7), and the BDD Lab / TestLab2 (Lecture 9). For each lab this guide gives: the assignment exactly as the handout states it, a concrete step-by-step walkthrough with code and commands, report-ready why/how justification sentences, a self-contained copy-paste reflection (the only place with [bracketed placeholders]), likely exam questions with answers, pitfalls plus what the examiner rewards, and the theory each lab exercises. Citations: lab handouts as (RefactLab p.1), (ActLab p.1), (TestLab1 p.1), (BDDLab p.1); decks as (Refactoring1 p.N), (HighLevelRefactoring p.N), (BetterCode p.N), (Actualization p.N), (Clean Architecture p.N), (OOPrinciples p.N), (Software Testing p.N), (BDD p.N); exam facts as (What-To-Expect p.N). Anything beyond the slides and handouts is flagged "(beyond slides — practical knowledge)". Companions: `90-exam-what-to-expect.md` (exam logistics), `91-exam-model-answers.md` (per-question model answers), `92-exam-copy-paste-library.md` (catalogs and short fill-in templates), `93-lab-deep-dives-1.md` (the Lecture 1–3 labs). This file goes deeper on what was actually *done* in each of these four labs and how to write about it.

Why these four labs dominate the exam report

The exam is a reflective report written during the exam from a handed-out template, and its biggest part is "reflecting on the practical work we did in labs," "explaining how we implemented things," and "explaining why we made certain technical decisions" — the lecturer stressed the exam is "not only about what you did, but especially: why you did it, how you did it" (What-To-Expect p.1). The lecturer's own list of important topics maps almost one-to-one onto these four labs:

- **Refactoring** — "How you refactored code, why you refactored it, which refactoring techniques/patterns you used," with the verbatim example question "What is meant by a refactoring pattern?" (What-To-Expect p.3). That is the Refactoring Lab, where you found smells in JHotDraw with SonarLint and removed them with catalog refactorings (RefactLab p.1).
- **Code Smells** — "How to identify code smells, examples of bad code structure, why certain code is problematic," with the verbatim example question "How do you identify a code smell?" (What-To-Expect p.3). Also the Refactoring Lab: smells are its first objective (RefactLab p.1).
- **Software Testing** — "How you applied software testing, why testing was important, what kind of tests you used" (What-To-Expect p.3), plus the lecturer's recalled exam question "How did you refactor the code and how did you apply the software testing" (What-To-Expect p.6). That is TestLab1 (JUnit 4 unit tests, boundaries, mocks, assertions — TestLab1 p.1) plus the BDD Lab (behaviour-level scenarios with JGiven and AssertJ — BDDLab p.1).
- **Clean Code & Architecture** — "Concepts from the Clean Code book, clean architecture concepts," applied to your own code with explained design decisions (What-To-Expect p.4–5). That is the Actualization Lab, whose portfolio literally asks for SOLID examples and a Clean Architecture explanation in the context of the CASE study (ActLab p.1).
- **Technical decision-making** — the lecturer's exemplar answer shape, "I used a switch statement instead of multiple if-statements because it improved readability and provided clearer structure" (What-To-Expect

p.4), is the decision → rejected alternative → concrete benefit pattern every section of this guide is written in.

Two more exam facts make these four labs the highest-value preparation targets. First, the GitHub repository is itself evaluated — "code changes, refactoring work, testing, pipeline setup, overall maintenance work" (What-To-Expect p.5) — and these labs *are* the refactoring and testing commits in that repository. Second, you may bring prewritten material and "copy/paste parts from your prepared material into the report" (What-To-Expect p.2), which is exactly what the reflection and justification sections below are for. Generative AI is the one banned aid (What-To-Expect p.2), so everything here is designed to work offline, as-is.

Refactoring Lab (Lecture 4) — smells to refactorings in JHotDraw

The assignment — what the handout asks

The Refactoring Lab handout ([RefactLab], one page) opens by defining the discipline you are about to practise: "Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior. Its heart is a series of small behavior preserving transformations... Since each refactoring is small, it's less likely to go wrong. The system is kept fully working after each refactoring" (RefactLab p.1). It then sets two objectives: (1) identify and understand Bad Code Smells, and (2) apply refactorings to get rid of Bad Code (RefactLab p.1).

The classwork is a five-item workflow (RefactLab p.1):

1. Make sure you have your own feature branch, using `git checkout -b your-feature development` — note the branch is cut from `development`, not from main.
2. Follow the feature branch workflow (the handout cites [GitHub flow]).
3. Install [sonarlint].
4. Find code smells in JHotDraw based on **your change request** and SonarLint — the handout adds " (shouldn't be a problem)", i.e. JHotDraw has plenty to find.
5. Apply one or more suitable Refactoring Patterns to get rid of the bad code smells, with a footnote pointing to the online catalog at refactoring.com/catalog.

The portfolio work — the part the exam report draws on — has four demands (RefactLab p.1): describe the **code smell that triggered your refactoring**, with Chapter 4 of [Ker05] (Kerievsky, *Refactoring to Patterns*) as the smell reference; describe **what you plan to change**; describe the **strategy** of the refactorings; and state **which of the refactorings from [Ker05] you applied and what the reasoning behind it was**. The handout closes: "Remember to describe the strategies and purpose of the Refactorings" (RefactLab p.1). Two details separate a precise answer from a vague one: smells are found *relative to your change request* (this is prefactoring for a concrete change, not random smell-hunting), and the refactorings you justify are Kerievsky's pattern-directed ones layered on top of SonarLint's automated detection (RefactLab p.1).

Step-by-step walkthrough — how to do it

The refactoring lab in practice runs as a measure → refactor → verify loop on your JHotDraw fork. Here is the full execution, step by step.

Step 1 — branch first. Cut a feature branch from `development` exactly as the handout commands (RefactLab p.1), and establish a green baseline before touching anything:

```
git checkout development
git pull
git checkout -b refactor-selection-tool development
mvn clean test        # green baseline: proof the suite passed BEFORE refactoring
```

The baseline run matters because behaviour preservation is verified *relative to* something — if you cannot show the tests were green before, a green run after proves nothing. (beyond slides — practical knowledge)

Step 2 — install SonarLint. SonarLint is an IDE plugin that statically analyses code and flags issues live as you type — the automated analog of the deck's "Symptoms of Bad Code" (RefactLab p.1). In IntelliJ it installs via Settings → Plugins → Marketplace → "SonarLint"; in Eclipse via the Eclipse Marketplace. Once installed, open the classes your change request touches and read its findings panel. (beyond slides — practical knowledge for the install mechanics)

Step 3 — scope the hunt with your change request. Do not scan all of JHotDraw. The handout says to find smells "based on your change request and sonarlint" (RefactLab p.1): the classes in your concept-location/impact-analysis result set are the hunting ground, because the point of this refactoring is to make *your* change easier — Fowler's "Refactor When You Add Function" trigger, which the course frames as **prefactoring** (Refactoring1 p.5).

Step 4 — name the smell and measure it. Cross-check three rulers on each suspect unit: Fowler's 22 smells for the *name* (Refactoring1 p.6–8), the SIG thresholds for the *number* — more than 15 lines of code violates G1 (BetterCode p.10), more than 4 branch points violates G2 (BetterCode p.16), a duplicated block of 6 or more lines violates G3 (BetterCode p.23), more than 4 parameters violates G4 (BetterCode p.28) — and SonarLint for tool confirmation. A typical JHotDraw find is a Long Method in a tool or figure class that mixes computation with drawing, announced by a comment explaining each phase — the Comments smell, "comments used to compensate for bad code" (Refactoring1 p.8). A second typical find is Duplicated Code: two figure classes computing near-identical bounds or drawing logic in parallel, a 6+-line G3 violation begging for an Extract Method followed by a pull-up into the shared superclass (Refactoring1 p.6; BetterCode p.23).

Step 5 — pick the refactoring from the catalog. The smell names the move. For a local mess use Fowler's low-level catalog (Extract Method for Long Method/Duplicated Code/Comments, Refactoring1 p.11; Introduce Parameter Object for Long Parameter List, Refactoring1 p.41); for a design-level smell use Kerievsky's smell→refactoring table from [Ker05] Chapter 4, which the portfolio explicitly asks you to cite — e.g. Switch Statements → Replace Conditional Dispatcher with Command, Long Method → Compose Method, Duplicated Code → Form Template Method (HighLevelRefactoring p.12–13).

Step 6 — apply the move in small steps, tests after each. A constructed example in the decks' style — substitute your own class and method names. Before, a Long Method in a selection tool (the comments are the smell's confession):

```
public void drawSelection(Graphics g) {
    // compute the bounds enclosing all selected figures
    Rectangle r = null;
    for (Figure f : selection) {
        Rectangle b = f.displayBox();
        r = (r == null) ? b : r.union(b);
    }
    if (r == null) return;
    // draw the dashed selection box
    g.setColor(Color.GRAY);
    g.drawRect(r.x - 2, r.y - 2, r.width + 4, r.height + 4);
}
```

After Extract Method (Refactoring1 p.11), each comment has become a method name and `drawSelection` reads as two intentions:

```
public void drawSelection(Graphics g) {
    Rectangle bounds = selectionBounds();
    if (bounds == null) return;
    drawDashedBox(g, bounds);
}

private Rectangle selectionBounds() { /* the loop, unchanged */ }

private void drawDashedBox(Graphics g, Rectangle r) { /* the drawing, unchanged */ }
```

For a type-switching smell, the canonical JHotDraw-shaped move is Replace Conditional with Polymorphism (Refactoring1 p.34): a `switch` that branches on a figure kind becomes an overridden method on each `Figure` subclass, so adding a new figure type means adding a class instead of editing every switch — exactly how JHotDraw's own `Figure` hierarchy (e.g. `RectangleFigure`, `EllipseFigure`) replaces type-switching over shape kinds (Refactoring1 p.34; JHotDraw's pattern roster is course context, not on the lab sheet).

Use the IDE's automated refactorings (Refactor → Extract Method, Rename, Move) rather than hand-editing — Kerievsky's **Automation First** heuristic: "manual refactorings are dirt roads, automated refactorings are highways" (HighLevelRefactoring p.20). After *each* single move:

```
mvn test
git add -A
git commit -m "Extract Method: selectionBounds() out of drawSelection (Long Method, G1)"
```

One refactoring, one test run, one commit — this is the handout's "series of small behavior preserving transformations" made visible in the history (RefactLab p.1). (beyond slides — the one-commit-per-refactoring discipline is practical knowledge)

Step 7 — re-measure to prove the smell is gone. Re-run SonarLint on the file and re-count against the SIG thresholds: the extracted units should each be at or under 15 lines (G1), at or under 4 branch points (G2), with no duplicated block of 6+ lines (G3) (BetterCode p.10, 16, 23). The before/after numbers are portfolio gold — "the method went from 27 lines to three units of 15, 8 and 6 lines" is evidence, not opinion.

Step 8 — push and merge per GitHub flow. Push the branch, open a pull request into `development`, and merge after review/green build — the feature branch workflow the handout cites (RefactLab p.1):

```
git push -u origin refactor-selection-tool
# then open a pull request into development on GitHub
```

Step 9 — write the portfolio entry while it is fresh. Record: the smell (name + measurement + where), the plan, the strategy (which small steps in which order), the [Ker05] refactoring applied and the reasoning, and the proof of behaviour preservation (test runs, commit list) — the handout's four portfolio demands in order (RefactLab p.1).

Why & how — report-ready justifications

Standalone sentences for the refactoring lab in the decision → rejected alternative → concrete benefit pattern the lecturer wants (What-To-Expect p.4); each works as-is in a report paragraph.

- I refactored on a feature branch cut from `development` instead of committing to the mainline, because the branch isolates restructuring risk and keeps `development` releasable while my refactoring is in progress (RefactLab p.1).
- I scoped the smell hunt to the classes of my change request instead of sweeping the whole codebase, because refactoring pays off where change is about to happen — Fowler's "refactor when you add function" — and effort elsewhere is speculative (Refactoring1 p.5).
- I identified smells with SonarLint *and* the named catalogs rather than by gut feeling, because a named smell (Long Method, Duplicated Code) plus a measured threshold (15 lines, 4 branch points, 6 duplicated lines) turns "this code feels bad" into a verifiable claim (Refactoring1 p.6–8; BetterCode p.10, 16, 23).
- I applied small named catalog refactorings instead of a big-bang rewrite, because "since each refactoring is small, it's less likely to go wrong" and the system stays fully working between steps — a rewrite would have abandoned that safety entirely (RefactLab p.1).
- I used the IDE's automated refactorings rather than manual editing, following Kerievsky's Automation First heuristic, because the tool preserves behaviour mechanically while a manual edit preserves it only as reliably as my attention (HighLevelRefactoring p.20).
- I re-ran the test suite after every single refactoring instead of once at the end, because behaviour preservation must be verified per transformation — a failure after ten unverified steps cannot be attributed to any one of them (RefactLab p.1).
- I chose Replace Conditional with Polymorphism over extending the existing switch, because the switch forces every new figure type to edit shared dispatch code, while polymorphism makes the next type a pure addition — open for extension, closed for modification (Refactoring1 p.34; OOPrinciples p.6).
- I documented each refactoring under its catalog name instead of describing edits informally, because the shared vocabulary lets the examiner check my mechanics against the catalog without reading the whole diff (Refactoring1 p.9–58; RefactLab p.1).

Copy-paste reflection for the report

In the refactoring lab I removed code smells from the part of JHotDraw my change request touches, without changing observable behaviour. My change request, [change request], pointed me at [Class/package], and SonarLint plus a manual pass against Fowler's smell catalog turned up the trigger smell: [smell, e.g. Long Method], concretely [method] at [N] lines and [M] branch points against the guideline limits of 15 and 4 (BetterCode p.10, 16). My plan was to [what you planned to change], and my strategy was a sequence of small behaviour-preserving steps rather than one rewrite: first [step 1, e.g. Extract Method on the bounds computation], then [step 2], each performed with the IDE's automated refactoring, each followed by a full `mvn test` run, and each committed separately — for example "[commit message]". From Kerievsky's catalog I applied [refactoring, e.g. Compose Method], whose reasoning fit my case exactly: [reasoning, e.g. the method mixed three levels of abstraction, and composing it out of same-level named steps made the policy readable at a glance]. I rejected the alternative of [rejected alternative, e.g. leaving the comments to explain the phases], because a comment compensates for unclear code instead of fixing it (Refactoring1 p.8). The measurable outcome: [unit] went from [N] lines / [M] branch points to [n]/[m], SonarLint reports no finding on the file, and the test suite was green before and after every step — which is what makes this refactoring, not editing. The branch merged to `development` through a pull request, so the repository history shows the whole transformation step by step.

Likely exam questions about this lab — with answers

What is meant by a refactoring pattern? A refactoring pattern is a named, catalogued, behaviour-preserving code transformation: a reusable recipe that says when a structural problem occurs, what sequence of edits fixes it, and why the result is better. Fowler's catalog groups about fifty such moves — Extract Method,

Move Method, Replace Conditional with Polymorphism — into seven categories (Refactoring1 p.9), and each entry follows Alexander's three-part pattern rule of context, problem and solution (HighLevelRefactoring p.7). In the lab I applied catalog refactorings rather than ad-hoc edits because each named move has known mechanics and a known risk profile (RefactLab p.1).

How do you identify a code smell? By symptom, by measurement, and by tool. Symptom: match the code against the named catalog — duplicated structure, a method too long, a class with many unrelated reasons to change, a switch on type codes (Refactoring1 p.6–8). Measurement: check the SIG thresholds — more than 15 lines violates G1, more than 4 branch points violates G2, a 6+-line duplicated block violates G3, more than 4 parameters violates G4 (BetterCode p.10, 16, 23, 28). Tool: SonarLint flags many of these automatically in the IDE (RefactLab p.1). A smell is a symptom, not a bug — the code may run perfectly and still smell.

How did you refactor the code and how did you apply the software testing? (a question the lecturer recalled verbatim from an example exam report, What-To-Expect p.6.) Answer in two halves, each as decision → alternative → benefit. Refactoring half: name the trigger smell with its measurement, the catalog refactoring applied, and the small-steps strategy — e.g. "the method violated Long Method/G1 at 27 lines; I applied Extract Method in three IDE-automated steps rather than rewriting, so the system stayed working throughout" (RefactLab p.1; Refactoring1 p.11). Testing half: the suite is what made the refactoring safe — green baseline before, full run after every step — and TestLab1's unit tests (best case, boundaries, mocks for dependencies) are how the feature itself was verified (TestLab1 p.1). The two labs interlock: refactoring "requires good testsuite" and testing's third TDD step *is* refactoring (Software Testing p.48).

Which refactoring did you apply and why that one? Name the smell first, then the move it indexes: the smell→refactoring tables exist precisely so the trigger justifies the choice — Long Method → Extract Method or Compose Method; Duplicated Code → Extract Method/Form Template Method; Switch Statements → Replace Conditional with Polymorphism or Replace Conditional Dispatcher with Command (Refactoring1 p.6–8; HighLevelRefactoring p.12–13). The reasoning sentence the portfolio asks for is exactly this pairing plus the benefit (RefactLab p.1).

How did you make sure the behaviour did not change? Three mechanisms: a green test baseline before starting, a full test run after every single small step, and IDE-automated transformations that preserve behaviour mechanically (RefactLab p.1; HighLevelRefactoring p.20). Behaviour preservation is the definitional constraint of refactoring — "the observable behavior of the software should not be changed" (Refactoring1 p.3) — so it must be verified, not assumed.

What is the difference between Fowler's refactorings and Kerievsky's? Fowler's are low-level: small, mechanical, single-step moves like Extract Method, often tool-automatable. Kerievsky's are high-level composites: planned sequences of low-level moves that introduce a whole design pattern — Replace Conditional Logic with Strategy, Move Embellishment to Decorator (HighLevelRefactoring p.12–16). "Design patterns are the word problems of the programming world; refactoring is its algebra" (HighLevelRefactoring p.9): you reach the pattern by small steps instead of designing it up front.

When should you refactor? At four trigger moments: when you add function (clean the area first so the new code fits), when you fix a bug (unclear code hides bugs), during code review, and per the Rule of Three — refactor the third time you do something similar, so you neither over-engineer the first occurrence nor let duplication spread (Refactoring1 p.5). In the change process this splits into prefactoring (before actualization, making room) and postfactoring (after, cleaning up).

Why is refactoring worth the time if it adds no functionality? Four reasons from the deck: it improves the design (which otherwise decays change by change), it makes software easier to understand (code is read far more than written), it helps you find bugs (clarifying structure surfaces hidden defects), and it helps

you program faster — the time spent is repaid by cheaper subsequent changes, so refactoring is an investment in velocity, not a tax (Refactoring1 p.4).

Pitfalls, mistakes, and what the examiner looks for

Mistakes that cost marks in the refactoring write-up:

- **Describing the edit without naming the smell.** "I split a big method" earns less than "the method violated Long Method and G1 at 27 lines, so I applied Extract Method" — the handout explicitly asks for the smell that *triggered* the refactoring (RefactLab p.1).
- **Mixing the vocabularies.** "Long Method violates G3" is wrong — G3 is duplication; Long Method breaks G1 (BetterCode p.10, 23). Keep Fowler smell names, SIG guideline numbers and Kerievsky refactoring names straight.
- **Claiming behaviour preservation without evidence.** If no test ran between steps, you restructured *hopefully*, not safely. The examiner wants the test that proves preservation — name the suite and when it ran.
- **Big-bang restructuring sold as refactoring.** One giant commit contradicts "a series of small behavior preserving transformations" (RefactLab p.1); the commit history is part of the graded repository (What-To-Expect p.5).
- **Calling refactoring "optimization."** Refactoring targets understandability and modifiability, explicitly not runtime performance (Refactoring1 p.3).
- **Smell-hunting without a change request.** The lab anchors smells to *your* change (RefactLab p.1); free-floating cleanup reads as not understanding prefactoring.
- **Over-applying patterns.** Abstraction with no current user is itself a smell (Speculative Generality, Refactoring1 p.7), and Kerievsky's Refactoring Directions table includes moves *away* from patterns, like Inline Singleton (HighLevelRefactoring p.21–22). The test is the triggering smell: no smell, no refactoring.

What elevates an answer: naming the smell *and* its measurement; citing the catalog refactoring by name with its strategy and purpose (the handout's closing demand, RefactLab p.1); pointing at the commit or pull request where the step-by-step transformation is visible; and quoting the before/after numbers that prove the smell is gone. The lecturer's grading pattern is decision → rejected alternative → benefit (What-To-Expect p.4) — every refactoring you report should carry all three.

Theory links

Lecture concepts this lab exercises, with deck citations:

- **The definition and constraint of refactoring** — internal-structure change, observable behaviour unchanged (Refactoring1 p.3); the lab's introduction restates it verbatim (RefactLab p.1).
- **Rajlich's change mini-cycle placement** — this lab is the **prefactoring** phase when done before your feature change (make room) and **postfactoring** when done after (clean up the debt the change introduced); same techniques, different position relative to actualization (Refactoring1 p.5 for the add-function/code-review triggers; course-process framing).
- **The 22 code smells** as the trigger language (Refactoring1 p.6–8) and Kerievsky's additional smells — Conditional Complexity, Oddball Solution, Solution Sprawl, Indecent Exposure, Combinatorial Explosion (HighLevelRefactoring p.12–13).
- **The seven refactoring categories** and the catalog moves (Refactoring1 p.9–58).
- **The SIG maintainability guidelines** as the measuring ruler — G1 unit size, G2 complexity, G3 write code once, G4 small interfaces, through G10 "leave no code smells behind" (BetterCode p.10–57).

- **High-level/composite refactoring** toward design patterns, the algebra metaphor, Automation First and Client First (HighLevelRefactoring p.8–20).
- **Verification as the safety net** — automated tests are what make behaviour preservation checkable (BetterCode p.53, Guideline 9), connecting this lab forward to TestLab1.
- **Version control discipline** — feature branch from `development`, GitHub flow, pull-request merge (RefactLab p.1), connecting back to the Git/CI labs and the graded repository (What-To-Expect p.5).

Actualization Lab (Lecture 5) — SOLID and Clean Architecture in the CASE study

The assignment — what the handout asks

The Actualization Lab handout ([ActLab], one page) opens with a three-part definition that is the most quotable sentence of the whole lab: "The actualization phase consists of the implementation of the new functionality, its incorporation into the old code, and change propagation that seeks out and updates all places in the old code that require secondary modification" (ActLab p.1). Read that as three distinct jobs — *implement* the new code, *incorporate* it (plug it into the old code), and *propagate* (hunt down every secondary modification the plug-in forces) — because the deck structures the whole lecture around exactly those three (Actualization p.1, p.8–11, p.16–21).

The objectives are two (ActLab p.1): "Understand and explain Clean Architecture in context of Actualization" and "Understand and explain Clean Code Principles in context of Actualization." The portfolio work is two deliverables (ActLab p.1): "Provide examples of the SOLID principles in context of the CASE study" and "Explain Clean Architecture in context of the CASE Study."

Three reading notes on the handout itself. First, unlike the other labs it has **no Classwork section** — no tool to install, no commands to run. The "doing" of this lab is the actualization of your own feature on your branch; the graded output is *explanation*, which makes it the most exam-shaped lab of the course: its portfolio items are literally report paragraphs. Second, the portfolio list ends with a third bullet that is **empty** — the sheet stops mid-list, so only the two deliverables above are actionable; do not invent a third. Third, both portfolio items say "in context of the CASE study": a recited textbook definition of SRP or a redrawn circles diagram earns little — the marks are for pointing at *your* JHotDraw classes (ActLab p.1).

Step-by-step walkthrough — how to do it

Step 1 — size the change and choose the actualization technique. The deck's first decision: small changes are made *directly in the old code* — the worked example changes `Address`'s ZIP field from `zip[5]` to `zip[9]`, a one-class edit (Actualization p.2–4). Larger changes — anything adding a new responsibility — are *implemented separately as new classes, then incorporated*, accepting the ripple that incorporation triggers (Actualization p.5). The deck's own large example is the cashier-login change request: "Create a cashier login that will control the user log in with a username and password," solved by writing a new `Cashiers` class and wiring it into the launch path (Actualization p.12–14). Decide which kind your feature is and say so in the portfolio — the technique differs with size, and naming that decision is free marks.

Step 2 — implement the new functionality separately, applying SOLID as you write. Constructed example in the deck's style — substitute your feature. Suppose the change request adds a new figure type to JHotDraw. The polymorphic shape of the deck's Farm/`Pig` example (Actualization p.6–7) maps directly: the new class plugs under the existing abstraction, and no client changes.

```
// New code, written and unit-tested in isolation before incorporation
public class TriangleFigure extends AbstractFigure {

    private final FigureStyle style;   // DIP: abstraction injected, not constructed

    public TriangleFigure(FigureStyle style) {
        this.style = style;            // constructor injection (OOPrinciples p.13)
    }

    @Override
    public void draw(Graphics g) {     // Polymorphism: no switch anywhere
        style.apply(g);
        // triangle-specific drawing only - SRP: this class only renders a triangle
    }

    @Override
    public Rectangle displayBox() { /* triangle bounds */ }
}
```

While writing, apply the principles you will later cite: **SRP** — the class does one thing, render its figure, not also persist or validate (OOPrinciples p.4–5); **DIP** — collaborators arrive through constructor-injected abstractions, the `Payments` / `PaymentMethod` shape (OOPrinciples p.12–13); **LSP** — the subtype honours every contract of `Figure`, so no client needs an `instanceof` (OOPrinciples p.8–9).

Step 3 — incorporate: plug the new code into the old code. The deck gives incorporation three structural shapes (Actualization p.8–11): the new responsibility is **local** to an existing class (just edit that class); the new class joins an existing **composite** (the whole aggregates one more part); or the new class is a **new supplier** — an old client now points at it. Identify which shape yours is and name it. Adding `TriangleFigure` is the composite/polymorphic shape: registration with the tool palette or figure factory is the only old-code edit. The deck's `Cashiers` example is the new-supplier shape: the launch code (old client) now calls `login()` on the new class (Actualization p.13–14, p.11). Incorporation is usually a few lines — but those lines make old code inconsistent, which is what the next step repairs.

Step 4 — propagate the change until consistency returns. Change propagation is a mark-and-visit walk of the dependency graph, and the deck animates it in colours on the `taxCategory` example (Actualization p.16–21): the new class is incorporated into `item`; modifying `item` *marks* its neighbours `store` and `saleLineItem` as suspect; each marked class is inspected — `store` needs nothing, `saleLineItem` needs a secondary modification, which marks `sale`; `sale` changes, marking `register`; `register` needs nothing; no marked classes remain, so **propagation ends** (Actualization p.21). Run the identical discipline on your feature: after each modification, list the classes that depend on what you just touched, inspect each, and keep a propagation log — "class → inspected/modified → why" — which becomes portfolio evidence. Note the deck's warning that even *deleting* obsolete functionality propagates (Actualization p.22). Run `mvn test` after each propagation step and commit with a message naming the secondary change, so the repository shows the ripple being chased down. (beyond slides — the log table and per-step commits are practical knowledge)

Step 5 — score your impact prediction with precision and recall. Propagation is "the moment of truth" for the impact set you predicted in the Impact Analysis lab (Actualization p.23). Compare predicted-changed against actually-changed classes in a confusion matrix (Actualization p.24): **precision = TP/(TP+FP)** (how much of what you predicted was right) and **recall = TP/(TP+FN)** (how much of what actually changed you had predicted) (Actualization p.26–27). The deck's Ericsson Radio Systems data over 136 classes is the benchmark: TP=30, FP=0, TN=42, FN=64 — precision 100%, recall a sobering 32%, because the classes reached only *transitively* through the dependency graph are systematically invisible up front

(Actualization p.24–28). Computing your own two numbers and comparing the signature to Ericsson's is a high-mark portfolio move.

Step 6 — write the SOLID example bank from your own diff (portfolio deliverable 1). For each principle use a fixed three-part shape: slide-exact definition + your CASE-study element + why it limited actualization cost. Pre-assembled mapping to adapt: **SRP** — "a class should have only one reason to change" (OOPrinciples p.4); separate tool classes per gesture, like the `UserService` / `SecurityService` split (OOPrinciples p.5); a change request then names one class. **OCP** — "open for extension but closed for modification" (OOPrinciples p.6); the `Figure` hierarchy made your new figure pure addition, the `Validator` / `LoanApprovalHandler` move (OOPrinciples p.7). **LSP** — "subclasses should be substitutable for their base classes" (OOPrinciples p.8); every figure honours the base contract — a subtype throwing on a base operation is the `Ostrich.fly()` violation (OOPrinciples p.9). **ISP** — "many specific interfaces are better than a single, general interface" (OOPrinciples p.10); role-sized figure/handle/tool interfaces, the `IUser` / `IRole` / `IUserRole` split (OOPrinciples p.11). **DIP** — "depend upon abstractions, do not depend upon concretions" (OOPrinciples p.12); your injected `FigureStyle`, the `Payments` / `PaymentMethod` shape (OOPrinciples p.13).

Step 7 — explain Clean Architecture in the CASE study (portfolio deliverable 2). State the four characteristics of a successful architecture — testable, independent of UI, independent of database, independent of frameworks (Clean Architecture p.3–4). Draw or describe the concentric layers — Entities, Use Cases, Interface Adapters, Frameworks & Drivers — and the **Dependency Rule**: source-code dependencies point only *inward* (Clean Architecture p.5–8). Then map JHotDraw onto it: the drawing model (`Drawing`, `Figure`, `Handle`) plays Entities/Use Cases; tool and controller classes are Interface Adapters; Swing and file I/O are Frameworks & Drivers. Close with the lab-specific argument: the UI and the database are *details* that plug in (Clean Architecture p.16–18), so a "swap the GUI" or "change persistence" request actualizes entirely in the outer ring — Clean Architecture is the design property that keeps Step 4's propagation walk short.

Why & how — report-ready justifications

- I implemented my change as a new class incorporated into the old code rather than editing the old code inline, because the change added a new responsibility — the deck's rule is that larger changes are implemented separately and incorporated, while only small changes are made directly in old code (Actualization p.2–5).
- I incorporated the new class polymorphically under the existing abstraction instead of adding a type-switch to clients, because the polymorphic shape makes the next variant pure addition with zero client edits — the Farm/ `Pig` actualization shape and OCP (Actualization p.6–7; OOPrinciples p.6–7).
- I injected my class's collaborators through constructor parameters typed to abstractions instead of constructing concretions inside, because DIP turns "swap the supplier" change requests into non-events for the client (OOPrinciples p.12–13).
- I propagated the change by systematically marking and inspecting every dependent of each modified class instead of stopping at the first compile success, because propagation only ends when no inconsistent class remains — a clean compile does not prove consistency (Actualization p.16–21).
- I kept a propagation log and compared the actually-changed set against my impact-analysis prediction using precision and recall, because the Ericsson data shows prediction errs on the side of missing transitively-reached classes (100% precision, 32% recall), and measuring is the only way to know how badly (Actualization p.24–28).
- I kept my new logic in the model layer and out of the Swing classes, because the Dependency Rule makes UI a plug-in detail — business rules that do not depend on the UI survive a UI replacement untouched (Clean Architecture p.8, p.16–18).

- I gave each new class a single responsibility rather than one convenient God class, because a class with one reason to change keeps the next change request's impact set at one class — SRP is impact-analysis insurance (OOPrinciples p.4–5).
- I documented which incorporation shape my change used (new supplier, composite, or local) instead of describing the wiring informally, because naming the shape predicts the expected ripple and shows the change was planned, not improvised (Actualization p.8–11).

Copy-paste reflection for the report

In the actualization lab I implemented my change request, [change request], and incorporated it into JHotDraw following the three-part actualization definition: implementation, incorporation, and change propagation (ActLab p.1). Because the change [added a new responsibility / was a small local edit], I chose to [implement it separately as the new class [ClassName] and then incorporate it / make it directly in [Class]], which is the technique the lecture prescribes for changes of that size (Actualization p.2–5). While writing the new code I applied SOLID deliberately: [ClassName] has the single responsibility of [responsibility] (SRP), it plugs under the existing [abstraction] so existing clients needed no edits (OCP), and its collaborator [collaborator] is constructor-injected through an abstraction (DIP) — I rejected [rejected alternative, e.g. extending a switch in [Client]] because it would force every future variant to edit shared dispatch code. Incorporation took the shape of [a new supplier / a composite extension / a local responsibility] (Actualization p.8–11): the only old-code edit was [the wiring]. That edit made [neighbour class(es)] inconsistent, so I propagated: I marked each dependent of every modified class, inspected it, and either changed it or cleared it, ending when no marked classes remained — in total [N] classes inspected, [M] secondarily modified. Comparing that against my impact-analysis prediction gave precision [P] and recall [R], the same signature as the Ericsson study where transitively-reached classes are the ones missed (Actualization p.24–28). Architecturally, my change lived in [layer], and the Dependency Rule explains why the ripple never reached [untouched layer] (Clean Architecture p.8).

Likely exam questions about this lab — with answers

What is the actualization phase? Actualization is the phase of the software change process where the change is actually made: "the implementation of the new functionality, its incorporation into the old code, and change propagation that seeks out and updates all places in the old code that require secondary modification" (ActLab p.1). Its technique varies with change size — small changes are made directly in old code, larger ones are implemented separately and then incorporated (Actualization p.2–5). It sits after concept location, impact analysis and prefactoring, and before postfactoring, verification and conclusion.

How did you incorporate your change into the old code? Name the structural shape: the new responsibility was local to an existing class, joined an existing composite, or arrived as a new supplier that an old client now points at (Actualization p.8–11). Then give the concrete wiring — e.g. "my new class registered with the figure factory; that one registration line was the entire incorporation, because the polymorphic seam already existed." The deck's model is `Cashiers`: a new supplier wired into the launch path so login is now required (Actualization p.12–14).

What is change propagation and when does it end? Propagation is the systematic hunt for secondary modifications: every class you modify can make its neighbours inconsistent, so you mark dependents, inspect each, modify where needed (marking *their* neighbours), and repeat. It ends when no marked classes remain — the deck's `taxCategory` walk ends when `register` is inspected and needs nothing (Actualization p.16–21). Even deletions propagate (Actualization p.22).

How is the accuracy of impact analysis measured? With a confusion matrix comparing predicted-changed against actually-changed classes, summarised as precision = TP/(TP+FP) and recall = TP/(TP+FN) (Actualization p.24–27). In the Ericsson Radio Systems study of 136 classes, TP=30, FP=0, TN=42, FN=64: precision 100% but recall only 32% — everything predicted was right, but two-thirds of what actually changed was never predicted, because transitive dependencies are invisible up front (Actualization p.24–28). Propagation is therefore "the moment of truth" for the prediction (Actualization p.23).

Provide an example of a SOLID principle in the context of the CASE study. Use the three-part shape: definition, CASE element, benefit. Example for OCP: "classes should be open for extension but closed for modification" (OOPrinciples p.6); JHotDraw's `Figure` abstraction with its shape subtypes meant my new figure type was a pure addition — no client edited, exactly the `Validator` move that frees `LoanApprovalHandler` from concrete validators (OOPrinciples p.7); the benefit is that a new-feature change request became new code with near-zero propagation (Actualization p.5–7). Have one such answer ready per principle.

Explain Clean Architecture in the context of the CASE study. Concentric layers — Entities, Use Cases, Interface Adapters, Frameworks & Drivers — with the Dependency Rule that source-code dependencies point only inward (Clean Architecture p.5–8). In JHotDraw the drawing model (`Drawing`, `Figure`, `Handle`) is the Entities/Use-Cases core; tools and controllers are Interface Adapters; Swing and file I/O are Frameworks & Drivers. The payoff is the four characteristics — testable, independent of UI, database, and frameworks (Clean Architecture p.3–4) — so UI- or persistence-shaped change requests actualize in the outer ring only, the same argument that makes the database "a detail" behind an Entity Gateway (Clean Architecture p.16–18).

What is the difference between an architecture and a design pattern? An architecture is the system-wide structure — the layers, boundaries and dependency directions that every component obeys (Clean Architecture p.2, p.5–8) — while a design pattern is a reusable solution to a recurring, *localized* design problem (a Strategy here, an Observer there). Clean Architecture constrains the whole codebase via the Dependency Rule; a pattern restructures one collaboration. In actualization terms: architecture bounds how far any change can ripple, patterns shape how a single seam absorbs change.

Pitfalls, mistakes, and what the examiner looks for

- **Reciting definitions without the CASE study.** Both portfolio items end "in context of the CASE study" (ActLab p.1). A textbook SRP paragraph without a JHotDraw class name misses the deliverable; every principle needs a concrete class from *your* diff.
- **Treating actualization as just "writing the code."** The definition has three parts — implementation, incorporation, propagation (ActLab p.1). Reports that stop after "I implemented the feature" skip the two parts the lecture is actually about.
- **Stopping propagation at the first green compile.** Consistency, not compilation, ends propagation (Actualization p.21). Inspected-but-unchanged classes (the walk's grey nodes) belong in your account — they prove you looked.
- **Reversing the Dependency Rule.** Dependencies point *inward* — the UI depends on the use cases, never the reverse (Clean Architecture p.8). Drawing the arrows outward in the report is a classic giveaway of memorising the circles without the rule.
- **Confusing precision and recall.** Precision is correctness of the prediction (TP/(TP+FP)); recall is completeness (TP/(TP+FN)). Ericsson's failure mode was recall, not precision — they predicted nothing wrong but missed 64 classes (Actualization p.24–28).
- **Claiming a third portfolio deliverable.** The handout's third bullet is empty (ActLab p.1); there are exactly two deliverables.

- **SOLID name-dropping.** Listing all five acronym letters with one-liners earns less than two principles each carried through definition → your class → propagation benefit. Depth beats coverage here, because the lecturer grades reasoning (What-To-Expect p.4).

What elevates an answer: naming the incorporation shape; a propagation log with inspected *and* modified classes; your own precision/recall numbers compared against Ericsson's signature; SOLID examples that cite the slide definitions and your own classes in the same sentence; and a Clean Architecture explanation that ends with a ripple argument ("this is why my change never touched X") rather than with the diagram.

Theory links

- **Rajlich Ch.8 actualization** — the deck *is* Rajlich's chapter (the footer credits "Software Engineering: The Current Practice Ch. 8"): technique varies with size (Actualization p.1–5), polymorphism as an actualization mechanism (p.6–7), incorporation shapes (p.8–11), replacement of a class (p.15).
- **Change propagation and the ripple effect** — the mark-and-visit walk and its termination condition (Actualization p.16–21); deletion also propagates (p.22).
- **Impact analysis accuracy** — confusion matrix, precision/recall, the Ericsson study, and the invisibility argument for systematic under-estimation (Actualization p.23–28); links back to the Impact Analysis lab in `93-lab-deep-dives-1.md`.
- **SOLID** — history and one-liners (OOPrinciples p.2–3), the five worked violation/repair pairs (p.4–13), plus the supporting principles CRP (p.14) and the Law of Demeter (p.15), whose train-wreck call chains are exactly the invisible coupling that wrecks recall.
- **GRASP** — the nine responsibility-assignment patterns (OOPrinciples p.16–27) for "which class gets this responsibility?" follow-ups: Information Expert, Creator, Controller, Polymorphism, Indirection, Pure Fabrication, Protected Variations.
- **Clean Architecture** — four characteristics (Clean Architecture p.3–4), layers and the Dependency Rule (p.5–8), Boundary/Interactor/Request-Response data flow (p.9–15), the database as a detail behind the Entity Gateway (p.16–18).
- **Process placement** — actualization sits between prefactoring (the Refactoring Lab's "make room" half) and postfactoring/verification (TestLab1's territory), under continuous verification (Actualization p.1).

Testing Lab (Lecture 7) — JUnit 4 unit tests for your feature

The assignment — what the handout asks

The handout ([TestLab1], one page, titled "Testing") opens with a quotable definition: "unit testing is a software testing method by which individual units of source code — sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures — are tested to determine whether they are fit for use" (TestLab1 p.1). It adds that unit tests are "typically automated tests written and run by software developers"; that in procedural programming a unit is commonly "an individual function or procedure" while in OO programming "a unit is often an entire interface, such as a class, but could be an individual method"; and that "by writing tests first for the smallest testable units, then the compound behaviors between those, one can build up comprehensive tests for complex applications" (TestLab1 p.1).

Objectives: "Understand the importance of testing" and "Implement unit tests" (TestLab1 p.1). The classwork is five numbered steps (TestLab1 p.1):

1. "Add maven dependency to [JUnit4] if it is not already done."

2. "Create JUnit 4 tests for most important domain logic methods of your feature. Note: Swing and JUnit extensions often works best with JUnit 4."
3. "Write JUnit tests for best case scenario."
4. "Write JUnit tests for identified boundary cases" — with sub-rule 4(a), the lab's most quotable line: "A unit test should test a single code-path through a single method. When the execution of a method passes outside of that method, you have a dependency and should apply mocks/stubs to avoid the dependency, see [mockito.org]."
5. "Use JAVA Assertions to test invariants, see [JAVA Asserts]" — with 5(a) "Assertions should be used to check something that should never happen" and 5(b) "an assertion should stop the program from running, but an exception should let the program continue running."

Portfolio work: "At class level write unit tests of important business functionality of your selected Feature. Document how you have verified your Feature." (TestLab1 p.1). Two deliverables, then: the test classes (scoped to *business functionality*, not widgets) and a written verification account. The tool list the handout fixes is exact: Maven, JUnit 4 (with the Swing-compatibility note), Mockito via [mockito.org], Java assertions via [JAVA Asserts] — nothing else (TestLab1 p.1).

Step-by-step walkthrough — how to do it

Step 1 — add the JUnit 4 dependency to the POM. Lab step 1 verbatim (TestLab1 p.1). In your JHotDraw fork's `pom.xml`:

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
```

Then confirm the build runs tests: `mvn clean test`. (beyond slides — the exact coordinates and `scope` are practical knowledge; the lab only says "add maven dependency to [JUnit4]".) The version pin to JUnit **4** is deliberate, not legacy laziness: JHotDraw is a Swing application and "Swing and JUnit extensions often works best with JUnit 4" (TestLab1 p.1).

Step 2 — pick the targets: domain logic, not widgets. List the methods of your feature that compute, decide, or mutate model state. This is the deck's "test under the UI" strategy (Software Testing p.29): GUI inputs are "clicks, events, swipes" producing "App. states" — brittle to drive and hard to assert (Software Testing p.30) — so verify the logic *beneath* the Swing layer. If a listener does more than delegate, that is a testability smell: refactor so it delegates, then test the delegate. A natural JHotDraw target is the very method your refactoring lab produced — e.g. `selectionBounds()` extracted out of `drawSelection` — because extraction is what made it testable in isolation. (Constructed example; substitute your feature's classes.)

Step 3 — best case first. Lab step 3 (TestLab1 p.1), in Arrange-Act-Assert form with the precondition asserted before acting, exactly as `testLoginAdmin` asserts `assertFalse(libApp.adminLoggedIn())` before logging in (Software Testing p.33):

```
public class SelectionBoundsTest {

    private SelectionTool tool;

    @Before
    public void setUp() {                       // fresh fixture per test (Software Testing p.52)
```

```
        tool = new SelectionTool();
    }

    @Test
    public void boundsEncloseSingleSelectedFigure() {
        // Arrange + precondition
        Figure rect = new RectangleFigure(new Point(10, 10), new Point(40, 30));
        assertTrue(tool.selection().isEmpty());
        tool.select(rect);
        // Act
        Rectangle bounds = tool.selectionBounds();
        // Assert
        assertEquals(new Rectangle(10, 10, 30, 20), bounds);
    }
}
```

Use `@Before` / `@After` for a fresh fixture per test and `@BeforeClass` / `@AfterClass` only for expensive shared setup (Software Testing p.52); pick the most specific assertion — `assertEquals`, `assertNull`, `assertTrue` — so a red test is self-explanatory (Software Testing p.53).

Step 4 — boundary cases second. Lab step 4 (TestLab1 p.1). Identify each method's equivalence classes and test the *edges*, following the Queue's full/empty/wrap model whose six-call run yields `true, true, false, 6, 7, null` (Software Testing p.11) and the `book==null` defect test (Software Testing p.47):

```
@Test
public void boundsAreNullForEmptySelection() {        // empty edge
    assertNull(tool.selectionBounds());
}

@Test
public void boundsOfZeroSizeFigureAreItsOrigin() {    // degenerate-size edge
    tool.select(new RectangleFigure(new Point(5, 5), new Point(5, 5)));
    assertEquals(new Rectangle(5, 5, 0, 0), tool.selectionBounds());
}

@Test(expected = IllegalArgumentException.class)       // null defect test, p.47/p.55 idiom
public void selectingNullFigureIsRejected() {
    tool.select(null);
}
```

The `@Test(expected=...)` idiom and the try/`fail()`/catch alternative are both on the deck (Software Testing p.55). Best-case-only suites miss exactly where defects cluster — the edges.

Step 5 — isolate dependencies with mocks/stubs. Lab rule 4(a): a unit test covers "a single code-path through a single method"; when execution leaves the method, you have a dependency to double (TestLab1 p.1). With Mockito (Software Testing p.58–71), constructed example: an alignment command whose only collaborator is the `DrawingView` interface:

```
@Test
public void alignCommandRepaintsTheView() {
    DrawingView view = Mockito.mock(DrawingView.class);          // mock() (p.65)
    Mockito.when(view.selection())                                // stub a query (p.69)
           .thenReturn(Collections.singletonList(rect));

    new AlignLeftCommand(view).execute();

    Mockito.verify(view).repaint();                               // verify a command (p.69, 71)
}
```

Choose the lightest double that works: a **stub** holds predefined data for queries (Software Testing p.60); a **mock** records and verifies interactions for commands (Software Testing p.59); a **spy** wraps a real object and overrides a few methods (Software Testing p.72–73). Remember the limitations — Mockito cannot mock final classes, anonymous classes, or primitive types (Software Testing p.67) — and that `@Mock` / `@InjectMocks` fields need `MockitoAnnotations.initMocks(this)` (Software Testing p.65, p.75). Two slide typos not to copy: the fluent call is `Mockito.when(...)`, not `test.when(...)`, and the class is `Mockito`, not `Mokito` (Software Testing p.70–71, p.75).

Step 6 — encode invariants as Java assertions. Lab step 5 (TestLab1 p.1). Write a `checkRep`-style method bundling the representation invariant, modeled on the Queue's ring-buffer check (Software Testing p.19):

```
private void checkRep() {
    assert selection != null : "selection list must exist";
    Rectangle b = selectionBounds();
    assert selection.isEmpty() || (b.width >= 0 && b.height >= 0)
        : "non-empty selection must have non-negative bounds";
}
```

Obey the three rules: assertions are not error handling, must have no side effects, and must not be silly like `assert 1+1==2` (Software Testing p.18). Keep 5(a)/5(b) straight: an assertion checks what should *never* happen and stops the program; an exception handles expected errors and lets it continue (TestLab1 p.1). Java assertions are off by default — enable them when testing (Software Testing p.16): `mvn test -DargLine="-ea"`, or configure surefire's `argLine` in the POM. (beyond slides — the Maven flag is practical knowledge.)

Step 7 — run everything and document the verification. Run `mvn test` for the whole suite — the cheap, repeatable regression run that manual testing can never be (Software Testing p.81). Then write the portfolio's verification account: a simple table mapping scenario → test → result (main scenario, each boundary, each mocked dependency, the invariant), which is your Verification-phase evidence — "Document how you have verified your Feature" (TestLab1 p.1).

Why & how — report-ready justifications

- I pinned the project to JUnit 4 instead of JUnit 5, because JHotDraw is a Swing application and the lab states the Swing-related JUnit extensions work best with JUnit 4 — compatibility beat novelty (TestLab1 p.1).
- I tested the domain logic under the UI instead of driving the Swing widgets, because GUI inputs and outputs (clicks in, application states out) make tests slow and fragile, while the logic beneath is fast, deterministic, and unit-testable (Software Testing p.29–30).
- I wrote the best-case test first and the boundary tests second, because the happy path pins the contract and the edges — empty, zero-size, null, full — are where defects cluster, as the Queue's full/empty/wrap run demonstrates (Software Testing p.11; TestLab1 p.1).
- I replaced my method's external dependency with a Mockito mock instead of letting the test call the real collaborator, because a unit test should cover a single code-path through a single method, and the real dependency would have made the test an integration test with someone else's failure modes (TestLab1 p.1).
- I used a stub for queries and reserved `verify()` for commands instead of mocking everything, because over-specified interactions make tests fragile — canned data suffices when only the returned state matters (Software Testing p.59–60).
- I encoded my feature's invariants as `assert` statements rather than defensive if-throws, because an invariant violation is a bug that should stop the program at the point of corruption, while exceptions are

for expected errors the program should survive (TestLab1 p.1; Software Testing p.18).
- I automated the whole suite under Maven instead of relying on manual checks, because manual tests are too expensive to run often, while an automated suite runs on every change and gives immediate regression feedback (Software Testing p.81).
- I documented which test covers which scenario instead of just committing the test classes, because the lab's portfolio deliverable is the verification account, and a suite nobody can map to scenarios proves nothing to a reader (TestLab1 p.1).

Copy-paste reflection for the report

In the testing lab I verified the domain logic of my feature, [feature], with JUnit 4 unit tests. I added the JUnit 4 Maven dependency to the POM — JUnit 4 rather than 5 because JHotDraw is a Swing application and the Swing-related JUnit extensions work best with JUnit 4 (TestLab1 p.1). I deliberately tested under the UI: instead of driving Swing widgets, I targeted the model-level methods that carry the business rules — above all [method] in [Class] — because GUI-driven tests are slow and fragile while model tests are fast and deterministic (Software Testing p.29–30). I wrote the best-case test first: [best-case test, e.g. a single selected figure yields its enclosing bounds], asserting the precondition before acting. Then I added boundary tests for the edges I had identified: [boundary 1, e.g. empty selection returns null], [boundary 2, e.g. a zero-size figure], and a null-input defect test expecting [exception]. Where [method] depended on [collaborator], I kept the test to a single code-path by replacing the dependency with a Mockito [mock/stub]: I stubbed [query] with when(...).thenReturn(...) and verified [interaction] with verify(...), rejecting the alternative of calling the real [collaborator] because that would have coupled my unit test to its failure modes (TestLab1 p.1). I also encoded the invariant [invariant] as a Java assert in a checkRep-style method, run with assertions enabled — an assertion stops the program on a should-never-happen state, unlike an exception (TestLab1 p.1). The suite runs green under `mvn test`; my verification document maps each scenario to its test, which is how I can show — not just claim — that the feature is verified.

Likely exam questions about this lab — with answers

How did you apply the software testing? (the lecturer's own emphasized topic and recalled exam question, What-To-Expect p.3, p.6.) Structure the answer as the lab's five steps applied to your feature: JUnit 4 via Maven; unit tests at class level for the feature's domain logic, tested under the UI; best-case scenario first, then identified boundary cases; mocks/stubs wherever execution left the method under test, keeping each test a single code-path; Java assertions for invariants (TestLab1 p.1). Close with the verification document: which test covers which scenario, all green under `mvn test`.

Why was testing important for your change? Three course-grounded reasons. Refactoring and actualization are only safe relative to a green suite — behaviour preservation is checked, not hoped (Software Testing p.48; RefactLab p.1). Automated tests give immediate regression feedback on every later change, which manual testing is too expensive to provide (Software Testing p.81). And testing is the change process's Verification phase: the change is not done until tests demonstrate it works and broke nothing.

What kind of tests did you use? Unit tests at class level for domain logic (TestLab1 p.1), comprising best-case tests for the main scenario, boundary tests at the edges of each equivalence class, defect tests for null/illegal inputs, and interaction tests using Mockito doubles where the unit had dependencies. Plus Java assertions as always-on invariant checks inside the code, and — at behaviour level — the BDD lab's JGiven scenarios as acceptance tests (BDDLab p.1).

What is the difference between a mock and a stub? A stub holds predefined data to answer queries during the test — "the lightest and most static" double (Software Testing p.60). A mock records method calls

and lets you *verify* the interaction afterwards — "the most powerful and flexible" double (Software Testing p.59). Use a stub when the SUT needs an answer; use a mock when the SUT's *collaboration* is the thing under test. A spy is the in-between: a real object, partially overridden (Software Testing p.72–73).

Why can a passing test suite not prove the program is correct? Because "testing can demonstrate the presence of bugs, but not their absence" (Dijkstra, Software Testing p.6) — exhaustive testing is theoretically impossible since all mainstream languages inherit the halting problem's undecidability (Software Testing p.4–5). A green suite is evidence about the paths you exercised; residual bugs can hide in every path you did not. That is why equivalence partitioning and boundary analysis matter — they maximize assurance per test on a finite budget (Software Testing p.11–12).

When should you use an assertion instead of an exception? Use an assertion for a condition that should *never* happen if the code is correct — a broken invariant, a violated precondition between trusted internals — and let it stop the program at the point of corruption; use an exception for expected, recoverable errors a valid caller can cause, and let the program continue (TestLab1 p.1; Software Testing p.18). Two corollaries: assertions are not input validation (Rule 1), and they must have no side effects because they can be disabled (Rule 2, Software Testing p.18).

What is a boundary case, and which did you test? A boundary case sits at the edge of an equivalence class — empty/full, zero/maximum, null, first/last — where off-by-one and overflow defects cluster; the deck's model is the fixed-size Queue whose interesting run is exactly its boundaries: `true, true, false, 6, 7, null` (Software Testing p.11). Then list yours: [empty selection, zero-size figure, null figure] — one sentence each on why that edge could plausibly fail.

Pitfalls, mistakes, and what the examiner looks for

- **Testing through the GUI.** Driving Swing widgets where a model call would do contradicts the lab's "domain logic" instruction and the deck's test-under-the-UI strategy (TestLab1 p.1; Software Testing p.29–30).
- **Happy-path-only suites.** Lab step 4 *requires* boundary cases (TestLab1 p.1); a suite with no empty/null/edge tests reads as not understanding where defects live (Software Testing p.11).
- **Letting unit tests call real dependencies.** Rule 4(a) is explicit: execution leaving the method means a dependency, and dependencies get mocks/stubs (TestLab1 p.1). A "unit test" that hits storage or another subsystem is an integration test mislabeled.
- **Assertions as error handling.** Validating user input with `assert` violates Rule 1 and dies in production only when assertions happen to be enabled; expected errors get exceptions (Software Testing p.18; TestLab1 p.1).
- **Forgetting `-ea`.** Java assertions are off by default (Software Testing p.16); an invariant suite that never runs verifies nothing. Say in the report *how* assertions were enabled.
- **Treating green as proof.** "Presence, not absence" (Software Testing p.6) — claim coverage of named scenarios and boundaries, never "the feature is bug-free."
- **Copying slide typos.** `test.when(...)`, `Mokito`, `MokitoAnnotations` are slide misprints; the compiling forms are `Mockito.when(...)`, `Mockito`, `MockitoAnnotations` (Software Testing p.70–71, p.75).
- **No verification document.** The portfolio asks you to *document* how the feature was verified (TestLab1 p.1) — committed tests without a scenario-to-test map deliver only half the lab.

What elevates an answer: the scenario→test→result table; naming the equivalence classes and why each boundary was chosen; the stub-vs-mock decision argued per dependency; the invariant stated in one sentence plus the `assert` that encodes it; and the connection both backwards (these tests made the refactoring safe)

and forwards (this suite is what the CI pipeline runs on every push — the repository evidence the examiner grades, What-To-Expect p.5).

Theory links

- **Incompleteness of testing** — Turing, the halting problem, and Dijkstra's "presence, not absence" (Software Testing p.3–6): why the lab asks for *well-chosen* tests, not all tests.
- **The basic test model and triage** — inputs → SUT → outputs → oracle (Software Testing p.7), and the failure-triage tree (SUT? test? spec? platform?) with the Mars Climate Orbiter as the spec-defect archetype (Software Testing p.9).
- **Testing levels and kinds** — unit/integration/system, white-box vs black-box, differential, stress, random (Software Testing p.8); the lab works the unit level (TestLab1 p.1).
- **Equivalence partitioning and boundary-value analysis** — the Queue example and the equivalent-tests slide (Software Testing p.11–12) ground lab step 4.
- **Design for testability and assertions** — the eight-item testable-software checklist ("Assertions, Assertions Assertions !!!", Software Testing p.14), assertion rules and `checkRep` (p.18–19), enable/disable trade-offs (p.16–17, 20–21).
- **TDD** — test before implementation, red/green/refactor, "implement only as much code so that the test does not fail," and the Borrow Book worked example (Software Testing p.35–47); refactoring as TDD's third step ties back to the Refactoring Lab (p.48).
- **JUnit machinery** — lifecycle annotations, assertion API, suites, exception idioms (Software Testing p.52–55).
- **Test doubles and Mockito** — mock/stub/spy definitions, fluent API, limitations (Software Testing p.58–75).
- **Fragile tests** — interface/behavior/data/context sensitivity and their cures (Software Testing p.25–28), motivating both test-under-the-UI and owning your test data.
- **Acceptance and regression testing** — user-defined acceptance tests (p.78–79), manual-vs-automated verdict and regression (p.81), connecting forward to the BDD lab and the CI pipeline.

BDD Lab (Lecture 9) — user stories to JGiven scenarios

The assignment — what the handout asks

The handout is titled "[TestLab2] Behavior Driven Testing" (BDDLab p.1) — the course's second testing lab, building on TestLab1. Its introduction (headed "Introduction – User Stories and BBD"; the heading misspells BDD as "BBD" — write "BDD" in your own answers) defines user stories: "User stories are short and simple descriptions of capabilities written from the perspective of the person who desires the new capability" (BDDLab p.1). It gives **two** templates, not one: "As a [user type], I want [some goal] so that [some reason]" and the alternative "As a [user type], I want [some goal] because [why]" (BDDLab p.1).

The body is Figure 1, "Example of how to map User Story to BDD scenario" (BDDLab p.1) — three worked story→scenario rows: the calculator user ("add two numbers" → Given I have two numbers 500 & 500 / When I add them up / Then I should get result 1000), the Math teacher ("automate marks sorting… declare top 5" → Given a list of numbers / When I sort the list / Then the list will be in numerical order), and the QA engineer ("check a critical feature… smoke test" → Given I visit Google.com / When I type 'TestingWhiz' as a search string / Then I should get search results matching TestingWhiz).

The portfolio is three bullets (BDDLab p.1): "Map your User Stories to BDD Given-When-Then Scenarios"; "Use [JGiven] to automate your BDD Scenarios"; and "For domain specific assertions use the [AssertJ

library]. For Swing applications use the [AssertJ-swing] to automate the Scenarios." That fixes the whole tool chain: **user story → GWT scenario → JGiven → AssertJ (→ AssertJ-Swing for Swing)**. Note what the handout does *not* contain: no classwork section, no Maven/GitHub steps, no JHotDraw mention — its scope is purely the story-to-automated-scenario pipeline, and since JHotDraw is a Swing application, the AssertJ-Swing clause applies to you (BDDLab p.1).

Step-by-step walkthrough — how to do it

Step 1 — map your user story to Given/When/Then. Take the user story you wrote in the Change Request lab and convert it exactly as Figure 1 demonstrates (BDDLab p.1). Two disciplines the figure quietly teaches: the story's *motivation* ("so that…") drops out — the scenario verifies the capability, not the reason it was wanted; and the scenario instantiates the goal with **concrete data** (two numbers become "500 & 500", the expected result "1000") — a scenario is an example, not a restatement. For a JHotDraw change request "as a user, I want to move figures by dragging so that I can arrange my drawing," the scenario becomes: *Given a drawing with one rectangle at (10, 10), When the user drags the rectangle 50 pixels right, Then the rectangle's x-coordinate is 60.* One logical When per scenario; Given is set-up only; Then is where assertion lives (BDD p.5).

Step 2 — add JGiven and AssertJ to the build. (beyond slides — the handout names the tools, the coordinates are practical knowledge):

```
<dependency>
    <groupId>com.tngtech.jgiven</groupId>
    <artifactId>jgiven-junit</artifactId>
    <version>1.3.1</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.24.2</version>
    <scope>test</scope>
</dependency>
```

JGiven integrates with the JUnit you already have from TestLab1 ("easy to integrate into existing test infrastructures (JUnit, TestNG)", BDD p.20).

Step 3 — write the scenario as a JGiven test method. A JGiven scenario is a normal JUnit test whose calls *read* like the prose scenario — no separate `.feature` file (BDD p.10). Transfer of the deck's pancake pattern onto JHotDraw (constructed example — the deck's own code is the pancake, BDD p.10–16):

```
public class MoveFigureScenarioTest
        extends ScenarioTest<GivenADrawing, WhenUserDrags, ThenFigure> {

    @Test
    public void a_rectangle_can_be_dragged_to_a_new_position() {
        given().a_drawing_with_one_rectangle_at( 10, 10 );
        when().the_user_drags_the_rectangle_right_by( 50 );
        then().the_rectangle_x_coordinate_is( 60 );
    }
}
```

The snake_case matters: JGiven derives the report prose from method names — underscores become spaces, arguments are woven in — so `the_user_drags_the_rectangle_right_by(50)` prints as "the user drags the rectangle right by 50" (BDD p.10, p.17).

Step 4 — implement the three stage classes with state transfer. One stage per Given/When/Then is the convention; stages are JGiven's unique feature and its reuse mechanism (BDD p.11). State flows between them via annotated fields — `@ProvidedScenarioState` in the producer, `@ExpectedScenarioState` in the consumer, matched by type (BDD p.12–13):

```
public class GivenADrawing extends Stage<GivenADrawing> {

    @ProvidedScenarioState
    Drawing drawing;

    @ProvidedScenarioState
    RectangleFigure rectangle;

    public GivenADrawing a_drawing_with_one_rectangle_at( int x, int y ) {
        drawing = new StandardDrawing();
        rectangle = new RectangleFigure( new Point( x, y ), new Point( x + 30, y + 20 ) );
        drawing.add( rectangle );
        return this;                        // self-type chaining, the GivenIngredients pattern (BDD
p.14)
    }
}

public class WhenUserDrags extends Stage<WhenUserDrags> {

    @ExpectedScenarioState
    RectangleFigure rectangle;              // consumed from the Given stage

    @ProvidedScenarioState
    int newX;                               // produced for the Then stage

    public WhenUserDrags the_user_drags_the_rectangle_right_by( int dx ) {
        assertThat( rectangle ).isNotNull();   // sanity guard, the WhenCook pattern (BDD p.15)
        rectangle.moveBy( dx, 0 );
        newX = rectangle.displayBox().x;
        return this;
    }
}

public class ThenFigure extends Stage<ThenFigure> {

    @ExpectedScenarioState
    int newX;

    public void the_rectangle_x_coordinate_is( int expectedX ) {   // void: nothing chains after
Then (BDD p.16)
        assertThat( newX ).isEqualTo( expectedX );
    }
}
```

This is the pancake trio transplanted: Given *provides* the world (as `GivenIngredients` provides `ingredients`, BDD p.14), When *consumes* it, acts, and *provides* the result (the `WhenCook` consume-act-provide shape, BDD p.15), Then *consumes* the result and asserts with AssertJ in a `void` step (the `ThenMeal` pattern, BDD p.16). The self-type generic `Stage<GivenADrawing>` is what keeps the fluent chain statically typed so `given().a_drawing…().and()…` autocompletes (BDD p.14–16).

Step 5 — assert in the domain's language with AssertJ. The Then step's `assertThat(newX).isEqualTo(60)` is AssertJ's fluent API: type-specific `assertThat` factories over `AbstractAssert`, chainable like `assertThat(actual).contains("is").startsWith("This")` (BDD p.24–25). For richer checks, package a predicate as a reusable `Condition` applied with `.is(...)` / `.isNot(...)` (BDD

p.26–27), or — the lab's "domain specific assertions" bullet — build a custom assertion: subclass `AbstractAssert` for your domain type, subclass `Assertions` to add your `assertThat(Figure)` factory, then write `assertThat(rectangle).hasDisplayBoxAt(60, 10)` exactly as the deck's `assertThat(student).isInMiddleSchool()` recipe prescribes (BDD p.28; BDDLab p.1).

Step 6 — run and harvest the reports. `mvn test` runs the scenarios like any JUnit test; JGiven prints the console report — the scenario rendered back as Given/And/When/Then prose — and generates data for the HTML5 app report with its Successful/Failed/Pending counts, tags, and per-class navigation (BDD p.17–18). The deck notes JGiven ships Maven and Jenkins plugins, which is how the scenarios join the CI pipeline so every push re-verifies the behaviour (BDD p.20); the `jgiven-maven-plugin` bound to `mvn verify` writes the browsable report under `target/jgiven-reports/html`. (beyond slides — the plugin coordinates and output path are practical knowledge.)

Step 7 — automate GUI-level scenarios with AssertJ-Swing where the behaviour only exists in the UI. The lab's final clause: "For Swing applications use the [AssertJ-swing] to automate the Scenarios" (BDDLab p.1). AssertJ-Swing simulates real user interaction — including drag 'n drop — with reliable component lookup by type, name, or custom criteria, supports all JDK Swing components, embeds screenshots of failed GUI tests into HTML reports, works with JUnit, and can even detect violations of Swing's EDT threading rules (BDD p.29). In a When stage, that means performing the drag on the real canvas via a `FrameFixture` instead of calling `moveBy` on the model — slower but verifying the full wiring; keep model-level scenarios as the fast default and reserve GUI-level ones for behaviour that only exists in the UI. (beyond slides — the `FrameFixture` API name is practical knowledge.)

Step 8 — document the pipeline for the portfolio. For each user story record: the story (both template slots filled), its GWT scenario, the JGiven test class and stages, and the report line proving it green — the lab's three portfolio bullets made checkable (BDDLab p.1).

Why & how — report-ready justifications

- I expressed my acceptance criteria as Given/When/Then scenarios instead of prose requirements, because the three-part shape forces every behaviour to declare its context, trigger, and expected outcome — and once automated, the specification cannot silently drift from the code, since a stale scenario fails (BDD p.4–5).
- I chose JGiven over a plain-text framework like Cucumber, because classical frameworks carry an "Additional Maintenance Cost" — a `.feature` file and regex glue to keep in lockstep — while JGiven scenarios are pure Java with full IDE refactoring support, one artefact instead of two (BDD p.6–7, p.20).
- I split my scenarios into Given/When/Then stage classes instead of writing monolithic test methods, because stages are reusable across scenarios — my drawing-setup stage serves every figure scenario — directly attacking the code-duplication problem that makes test suites expensive to maintain (BDD p.3, p.11).
- I passed state between stages with `@ProvidedScenarioState` / `@ExpectedScenarioState` fields instead of parameters or globals, because the annotations keep stages independent and reusable while documenting exactly what each stage consumes and produces (BDD p.12–13).
- I asserted with AssertJ instead of bare JUnit assertions, because JUnit's built-ins are simplistic and the Hamcrest/Fest add-ons are stagnant or abandoned, while AssertJ is an actively maintained, near-complete superset whose fluent chains read like sentences — keeping even the Then step aligned with BDD's readability goal (BDD p.22–23, p.25).
- I wrote a custom domain assertion rather than exposing raw getters in every Then step, because pushing the ubiquitous language into the assertion API makes the check read in the domain's words and reusable

wherever that type is asserted (BDD p.28).
- I kept my scenarios at model level by default and used AssertJ-Swing only where behaviour exists solely in the GUI, because model scenarios are fast and deterministic while GUI automation — though it verifies the real wiring and can catch EDT violations — is slower and heavier (BDD p.29).
- I treated the generated JGiven report as the lab's deliverable evidence, because the executed scenario rendered as prose is living documentation a non-developer can read — the report, not the Java, is what proves the behaviour to an examiner (BDD p.9, p.17–18).

Copy-paste reflection for the report

In the BDD lab I turned my user story into an executable acceptance test. My story — "As a [user type], I want [goal] so that [reason]" — mapped to the scenario: Given [context with concrete data], When [single action], Then [expected outcome with concrete values], following the handout's Figure 1 discipline of instantiating the goal with concrete examples while the motivation drops out of the scenario (BDDLab p.1). I automated it with JGiven rather than a plain-text framework like Cucumber because classical frameworks keep the scenario in a separate feature file wired to step definitions — two artefacts to maintain in lockstep, the "additional maintenance cost" the lecture warns about — whereas a JGiven scenario is pure Java with full IDE support (BDD p.6–7). I implemented three stage classes, [GivenStage], [WhenStage] and [ThenStage], one per scenario part: the Given stage provides [state] via @ProvidedScenarioState, the When stage consumes it with @ExpectedScenarioState, performs [action] and provides [result], and the Then stage asserts [outcome] in a void step — state flows down the pipeline through the annotations with no parameter passing (BDD p.13–16). My assertions use AssertJ for readability; for [domain type] I [wrote a custom assertion / used a Condition], so the Then step reads in domain language: [example assertion] (BDD p.25–28). Because JHotDraw is a Swing application, I [used AssertJ-Swing to drive the real GUI for [scenario] / kept scenarios at model level and noted AssertJ-Swing as the GUI-level option] (BDDLab p.1; BDD p.29). Running `mvn test` renders the scenario back as prose in the JGiven report — living documentation that stays true because a stale scenario fails.

Likely exam questions about this lab — with answers

What is BDD, and how does it differ from TDD? BDD specifies and verifies software by its externally observable behaviour, written in a common domain language understandable by domain experts, defined collaboratively, executed like normal tests, and serving as living documentation (BDD p.4). TDD is the same test-first discipline pointed inward — developer-facing unit tests driving design — while BDD reframes it around behaviour in shared language so the same artefact serves developer, tester, and domain expert (course-frame contrast; the deck does not state it on a slide). In the lab the BDD layer sits above TestLab1's unit tests: units verify methods, scenarios verify the story.

How did you map your user story to a BDD scenario? Quote the story in the handout's template, then show the conversion: the goal becomes a concrete worked example — Given the starting context with real data, When the single triggering action, Then the expected outcome with expected values — exactly as Figure 1 maps "add two numbers" to Given 500 & 500 / When I add them up / Then result 1000 (BDDLab p.1). Point out the two disciplines: motivation drops out, and abstract goals become concrete data, because scenarios are examples, not restatements.

Why JGiven instead of Cucumber or JBehave? The classical frameworks keep scenarios in plain-text or markup files (Gherkin, wiki, HTML) glued to Java step definitions — which the deck stamps "Additional Maintenance Cost": two artefacts in two languages that must stay in lockstep (BDD p.6). JGiven is developer-friendly — the scenario *is* Java, so there is one artefact, full IDE refactoring, and compile-time checking (BDD

p.7, p.9). The honest trade-off: domain experts can read JGiven's reports but cannot author its scenarios, unlike a Gherkin `.feature` file (BDD p.20).

How do JGiven stages share state? Through annotated fields, matched by type, copied between stages at execution time: `@ProvidedScenarioState` marks a field the stage produces, `@ExpectedScenarioState` marks one it consumes, and `@ScenarioState` is bidirectional (BDD p.13). In the pancake example, `GivenIngredients` provides the ingredient list, `WhenCook` expects it and provides `dough` and `meal`, and `ThenMeal` expects `meal` and asserts on it (BDD p.14–16). The mechanism keeps stages decoupled and reusable — no parameter wiring between them.

Why AssertJ rather than JUnit's built-in assertions? JUnit's assertions were "underpowered from the start," which drove teams to mix in Hamcrest and Fest — but Fest looks abandoned and Hamcrest is stagnant, leaving "a confusion of JUnit, Hamcrest and Fest" (BDD p.22). AssertJ is actively maintained, a near-complete superset of all three, well designed, and easy to read — fluent chains like `assertThat(actual).contains("is").startsWith("This")` (BDD p.23, p.25). For domain-specific checks it extends cleanly via Conditions and custom assertions (BDD p.26–28).

What is living documentation? Documentation generated from the executed scenarios themselves, so it cannot drift from the code: JGiven renders each run as Given/When/Then prose in the console and as a browsable HTML5 report with Successful/Failed/Pending counts, tags, and search (BDD p.17–18). Because the report is regenerated from real executions, a behaviour that stops working shows up as a red row — the document is self-verifying, which is exactly what prose specifications can never be (BDD p.4).

How would you verify a change to a Swing GUI like JHotDraw at the GUI level? With AssertJ-Swing, per the lab's explicit instruction for Swing applications (BDDLab p.1): it simulates real user interaction including drag 'n drop, looks up components reliably by type/name/criteria, supports all JDK Swing components, embeds screenshots of failures into HTML reports, and can detect violations of Swing's EDT threading rules — a realistic bug class for a constantly-repainting drawing framework (BDD p.29). Model-level scenarios stay the fast default; GUI-level scenarios verify the wiring the model cannot.

Pitfalls, mistakes, and what the examiner looks for

- **Restating the story instead of exemplifying it.** A scenario without concrete data ("Given some numbers / When I add them / Then I get the sum") misses Figure 1's whole lesson — scenarios are worked examples with real values (BDDLab p.1).
- **Keeping the motivation in the scenario.** The "so that…" clause belongs to the story; the scenario verifies the capability. Carrying it over signals a mechanical conversion (BDDLab p.1).
- **Multiple unrelated Whens.** One scenario, one logical trigger; a scenario that acts three times and asserts once is three behaviours crammed into one report line (BDD p.5).
- **Assertion-free Givens and Whens — or assertion-heavy ones.** Set-up belongs in Given, action in When, verification in Then; the only assertions outside Then are sanity guards like `WhenCook`'s `isNotNull` checks (BDD p.15–16).
- **Forgetting the state annotations.** An `@ExpectedScenarioState` field with no upstream provider arrives `null` at runtime — the sanity guards exist precisely to fail fast on broken wiring (BDD p.13, p.15).
- **Claiming domain experts write JGiven scenarios.** They cannot — that is the deck's single decisive JGiven limitation; experts read the reports, developers author the Java (BDD p.20). Getting this backwards inverts the framework's defining trade-off.
- **Writing a `.feature` file for JGiven.** JGiven's whole point is that there is no separate plain-text artefact; pairing it with Gherkin files reintroduces the maintenance cost it exists to remove (BDD p.6–7, p.10).

- **Reproducing the handout's typos.** The heading's "BBD" and the slide's IDE inlay hint `expectedMeal:` are artefacts, not syntax — write "BDD" and plain `the_resulting_meal_is_a("pancake")` (BDDLab p.1; BDD p.16).

What elevates an answer: the full pipeline shown for one real story — story (template slots named) → scenario (concrete data) → test method (snake_case mirroring the prose) → three stages (annotations explained) → AssertJ assertion (ideally custom/domain) → green report line; the JGiven-vs-Cucumber trade-off argued both ways; and the TNG evidence cited for the maintenance claim — three years, up to 70 developers, over 3000 scenarios, readability and reuse greatly improved, maintenance cost reduced though with "no hard numbers" (BDD p.19).

Theory links

- **BDD's definition and motivation** — the four defining properties (domain language, collaboration, executed like tests, living documentation, BDD p.4) answering the five typical test issues (technical clutter, unclear intent, duplication, developer-only readability, no documentation value, BDD p.3).
- **Given/When/Then** — the canonical scenario structure and the pancake example (BDD p.5); ubiquitous language and executable specifications as the anti-drift mechanism.
- **Framework taxonomy** — classical plain-text frameworks (Cucumber, JBehave, Concordion, Fitnesse, RobotFramework) and their "Additional Maintenance Cost" (BDD p.6) versus developer-friendly frameworks (Spock, ScalaTest, Jnario, Serenity, JGiven) (BDD p.7).
- **JGiven mechanics** — scenarios as JUnit methods (BDD p.10), stage classes as the unique modularity/reuse feature (BDD p.11), state transfer annotations and the Given→When→Then data pipeline (BDD p.12–16), console and HTML5 reports with the pass/fail/pending status model (BDD p.17–18), strengths and the domain-experts-cannot-author limitation (BDD p.20), TNG's industrial evidence (BDD p.19).
- **AssertJ** — why it exists (the JUnit/Hamcrest/Fest confusion, BDD p.22–23), the fluent machinery (BDD p.24–25), Conditions (BDD p.26–27), custom domain assertions (BDD p.28), and AssertJ-Swing for GUI verification including EDT-violation detection (BDD p.29).
- **Process placement** — user stories arrive from Initiation (the Change Request lab), scenarios are written before the code as acceptance criteria, actualization makes them green, and JGiven's Maven/Jenkins plugins run them in CI — BDD is the course's operationalisation of the Verification phase (BDDLab p.1; BDD p.20; course-frame synthesis).
- **Relationship to TestLab1** — unit tests verify methods in isolation with mocks at the boundaries (TestLab1 p.1); BDD scenarios verify whole behaviours in the user's language; together they cover the verification ladder from unit to acceptance (Software Testing p.8, p.78–79).